

DEALENGINE SOURCE CODE

Core Acquisitions Underwriting & CRM Webhook Pipeline — Redacted Showcase Edition

■ File: /schema.py (Pydantic Data Schemas)

```
from pydantic import BaseModel, Field
from typing import List, Optional

class PropertySummary(BaseModel):
    address: str
    sqft: int
    asking_price: float
    estimated_arv: float

class UnderwritingMetrics(BaseModel):
    rehab_tier: str = Field(description="Cosmetic, Standard, or Full Gut")
    rehab_cost_per_sqft: float
    total_estimated_rehab: float
    target_wholesale_fee: float
    max_allowable_offer_mao: float
    deal_spread: float
    is_viable_deal: bool

class OutreachAssets(BaseModel):
    suggested_opening_offer: float
    seller_sms_script: str
    seller_email_script: str

class DealEvaluationResponse(BaseModel):
    status: str = Field(description="SUCCESS, INCOMPLETE_DATA, or HIGH_RISK_DEAL")
    property_summary: PropertySummary
    underwriting_metrics: UnderwritingMetrics
    outreach_assets: OutreachAssets
    audit_flags: Optional[List[str]] = []

class SEOKeywordOpportunity(BaseModel):
    keyword: str
    search_volume: int
    keyword_difficulty: int = Field(description="Difficulty rating 0 to 100")
    cpc: float
    intent: str
    strategic_value: str = Field(description="Why this is a high-value term to target")

class LandingPageIdea(BaseModel):
    title: str
    slug: str
    target_keyword: str
    main_cta: str = Field(description="Primary lead magnet or call to action")
    suggested_structure: List[str] = Field(description="Structural sections of the landing page")

class BlogTopic(BaseModel):
    title: str
    target_keyword: str
    primary_intent: str
    key_talking_points: List[str]
    lead_magnet: str

class BacklinkCampaignIdea(BaseModel):
    domain: str = Field(description="The source referring domain or category (e.g. rentcast.io, biggerpockets.com)")
    authority_score: int = Field(description="Domain authority score from the backlink data (e.g. 70, 51)")
    strategic_angle: str = Field(description="Campaign angle to secure a backlink (e.g. affiliate partnership, calculator embed)")
    anchor_text_suggestion: str = Field(description="Optimal anchor text linking back to our platform")

class SEOStrategyResponse(BaseModel):
    executive_summary: str = Field(description="Overview of the organic SEO opportunity based on competitor data")
    top_keyword_opportunities: List[SEOKeywordOpportunity]
    suggested_landing_pages: List[LandingPageIdea]
    suggested_blog_topics: List[BlogTopic]
    suggested_backlink_campaigns: List[BacklinkCampaignIdea]
```


■ File: /main.py (Production FastAPI Entrypoint)

```
import os
import time
import httpx
import json
from typing import Optional
from dotenv import load_dotenv
from fastapi import FastAPI, HTTPException, Security, Depends, Query, BackgroundTasks, status
from fastapi.security import APIKeyHeader
from fastapi.responses import RedirectResponse
from pydantic import BaseModel, Field
from openai import OpenAI

from schema import DealEvaluationResponse

load_dotenv()

app = FastAPI(
    title="Real Estate Deal Evaluation & GoHighLevel API",
    version="1.0.0",
    description="Deterministic JSON micro-service for ARV, MAO, rehab tiers, outreach scripts, and GHL CRM integration."
)

# API Security Setup
API_KEY_NAME = "X-API-Key"
api_key_header = APIKeyHeader(name=API_KEY_NAME, auto_error=False)

is_gemini = api_key and api_key.startswith("AQ.")
base_url = "https://generativelanguage.googleapis.com/v1beta/openai/" if is_gemini else None
model_name = "models/gemini-3.5-flash-lite" if is_gemini else "gpt-4o-mini"

client = OpenAI(api_key=api_key, base_url=base_url)

TOKEN_STORE = {}
GHL_CLIENT_ID = os.getenv("GHL_CLIENT_ID")
GHL_CLIENT_SECRET = os.getenv("GHL_CLIENT_SECRET")
GHL_REDIRECT_URI = os.getenv("GHL_REDIRECT_URI")
GHL_API_BASE = "https://services.leadconnectorhq.com"

class PropertyEvaluationRequest(BaseModel):
    property_input: str = Field(..., example="Address: 1244 Maplewood Dr, Indianapolis, IN | SqFt: 1850 | Asking: $165,000 | ARV: $260,000 | Condition: Standard rehab needed.")

class GHLWorkflowPayload(BaseModel):
    locationId: str = Field(..., example="loc_xyz123")
    contactId: str = Field(..., example="contact_abc456")
    property_address: Optional[str] = None
    asking_price: Optional[str] = None
    arv: Optional[str] = None
    sqft: Optional[str] = None
    condition: Optional[str] = "Standard rehab needed"

@app.post("/v1/evaluate", response_model=dict, dependencies=[Depends(verify_api_key)], tags=["Core Analysis"])
async def evaluate_property(request: PropertyEvaluationRequest):
    start_time = time.time()
    try:
        with open("system_prompt.txt", "r", encoding="utf-8") as f:
            system_prompt = f.read()

        response = client.beta.chat.completions.parse(
            model=model_name,
            messages=[
                {"role": "system", "content": system_prompt},
                {"role": "user", "content": f"Evaluate this property:\n{request.property_input}"}
            ],
            response_format=DealEvaluationResponse,
            temperature=0.1
        )
```

```

latency = round(time.time() - start_time, 3)
output_data = response.choices[0].message.parsed.model_dump()

telemetry = {
    "status": "success",
    "latency_seconds": latency,
    "prompt_tokens": response.usage.prompt_tokens,
    "completion_tokens": response.usage.completion_tokens,
    "total_tokens": response.usage.total_tokens
}

    return {"data": output_data, "telemetry": telemetry}
except Exception as e:
    raise HTTPException(status_code=500, detail=str(e))
# =====
# 2. OAUTH 2.0 AUTHORIZATION HANDLERS
# =====

@app.get("/oauth/authorize", tags=["GHL Integration"])
def authorize_ghl():
    """Redirects agency admins to GHL consent page when installing the app."""
    scopes = "contacts.readonly contacts.write workflows.readonly"
    auth_url = (
        f"{GHL_API_BASE}/oauth/chooselocation?"
        f"response_type=code&"
        f"client_id={GHL_CLIENT_ID}&"
        f"redirect_uri={GHL_REDIRECT_URI}&"
        f"scope={scopes}"
    )
    return RedirectResponse(auth_url)

@app.get("/oauth/callback", tags=["GHL Integration"])
async def oauth_callback(code: str = Query(...)):
    """Receives auth code from GHL, exchanges it for access/refresh tokens."""
    payload = {
        "client_id": GHL_CLIENT_ID,
        "client_secret": GHL_CLIENT_SECRET,
        "grant_type": "authorization_code",
        "code": code,
        "redirect_uri": GHL_REDIRECT_URI,
    }

    headers = {"Content-Type": "application/x-www-form-urlencoded"}

    async with httpx.AsyncClient() as http_client:
        response = await http_client.post(
            f"{GHL_API_BASE}/oauth/token",
            data=payload,
            headers=headers
        )

        if response.status_code != 200:
            raise HTTPException(
                status_code=400,
                detail=f"Token exchange failed: {response.text}"
            )

        token_data = response.json()
        location_id = token_data.get("locationId")

        # Save tokens keyed by locationId
        TOKEN_STORE[location_id] = {
            "access_token": token_data.get("access_token"),
            "refresh_token": token_data.get("refresh_token"),
            "expires_in": token_data.get("expires_in")
        }

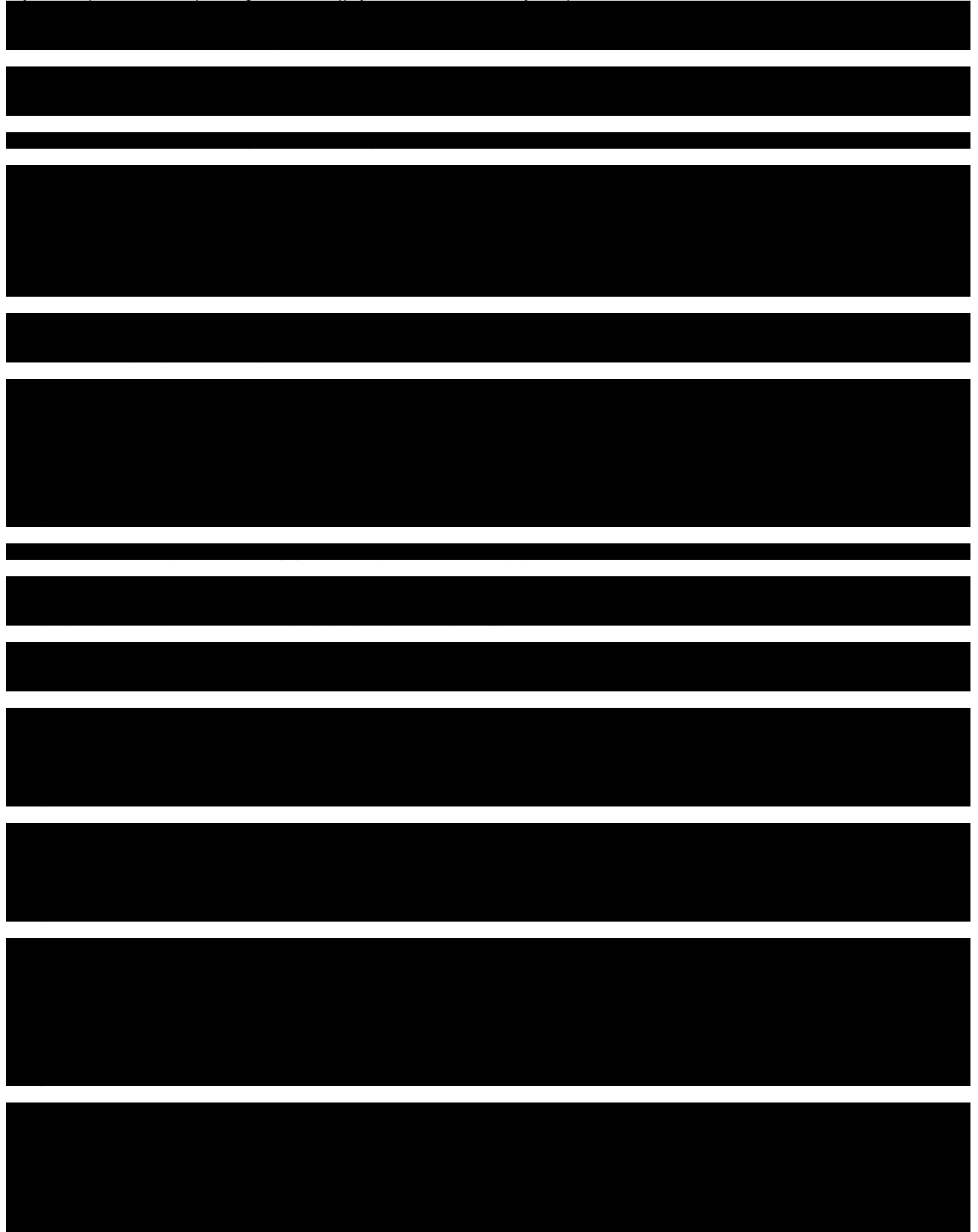
    return {
        "status": "success",
        "message": "App installed successfully in GoHighLevel!",
        "location_id": location_id
    }

# =====
# 3. GHL WEBHOOK/WORKFLOW ACTION BACKGROUND PROCESSOR

```

=====

```
async def process_and_update_ghl_contact(payload: GHLWorkflowPayload):
```



```

@app.post("/v1/ghl/workflow-action", tags=["GHL Integration"])
async def ghl_workflow_action(payload: GHLWorkflowPayload, background_tasks: BackgroundTasks):
    """
    Endpoint triggered by GoHighLevel Workflows.
    Responds immediately (200 OK) to prevent workflow timeouts and processes analysis in background.
    """
    background_tasks.add_task(process_and_update_ghl_contact, payload)
    return {"status": "queued", "message": "Deal evaluation started for GHL workflow."}

# =====
# 4. HEALTH CHECK DIAGNOSTIC ENDPOINT
# =====

@app.get("/health", tags=["Diagnostics"])
def health_check():
    return {"status": "ok", "service": "DealEngine API"}

```

■ File: /test_engine.py (Underwriting Helper Script)

```
import json
import os
import time
from dotenv import load_dotenv
from openai import OpenAI
from schema import DealEvaluationResponse

load_dotenv()

# Smart auto-detection of Gemini vs OpenAI keys to ensure zero-config execution
api_key = os.getenv("OPENAI_API_KEY")
is_gemini = api_key and api_key.startswith("AQ.")

base_url = "https://generativelanguage.googleapis.com/v1beta/openai/" if is_gemini else None
model_name = "models/gemini-3.5-flash-lite" if is_gemini else "gpt-4o-mini"

client = OpenAI(api_key=api_key, base_url=base_url)

def evaluate_deal_api(property_input: str):
    """
    Evaluates a property input using the OpenAI API.
    """
    # ... (rest of the function code) ...

if __name__ == "__main__":
    sample_input = "Address: 1244 Maplewood Dr, Indianapolis, IN | SqFt: 1850 | Asking: $165,000 | ARV: $260,000 | Condition: Standard rehab needed."
    res = evaluate_deal_api(sample_input)
    print(json.dumps(res, indent=2))
```

[REDACTED]

[REDACTED]

[REDACTED]